RBAC and ABAC

who can do what, on which thing, when?

Watch the 4-min video

In one line: RBAC grants permissions through roles and suits coarse, stable access; ABAC writes rules over attributes of user, resource and context; ReBAC (Zanzibar) derives access from relationships like “Bob views the folder this doc is in”.

1 Who can do what, on which thing?

A user can edit a document. Can they share it? Can their manager see it? Can someone in another region open it? Every product ends up asking these questions. Lesson 39 said authorization happens on every request; this lesson is about **how you model the rules**, which decides whether they stay simple or turn into special cases that leak data or lock people out.

2 RBAC and where it breaks

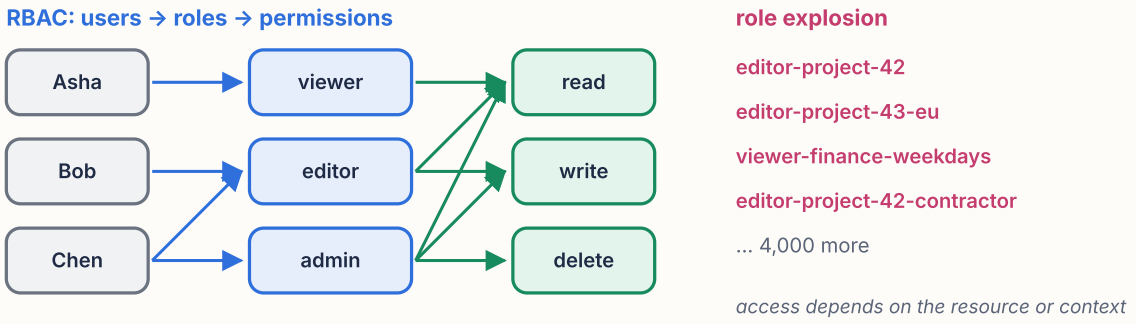


Figure 1. RBAC is a two-hop lookup. When access depends on *which* project, region or time, teams mint ever more specific roles.

Role-based access control: users get roles, roles carry permissions, and a check asks “does any of this user's roles grant `orders:write`?” It's simple, fast and easy to audit, ideal for internal tools and admin panels (Kubernetes RBAC, AWS IAM groups). It breaks when access depends on the **specific resource** (editor of *which* project?) or on **context** (business hours, region, device): that's **role explosion**.

3 ABAC: rules over attributes

Attribute-based access control (NIST SP 800-162) evaluates rules over attributes of the **subject** (department, clearance), the **resource** (owner, department, classification), the **action**, and the **environment** (time, IP, device posture). Example in Cedar:

```
permit(principal, action == Action::"edit", resource) when { principal.dept == resource.dept && context.managed_device };
```

Flexible, with no role explosion; but rules are harder to read, test and explain (“why was I denied?”).

4 Policy engines

Don't scatter `if` statements through the code. A **policy decision point** evaluates policy-as-code; services, the **enforcement points**, just ask “is this allowed?” **Open Policy Agent** (CNCF, Rego language)

is common for Kubernetes admission and API checks; **Cedar** (AWS, used by Amazon Verified Permissions) is designed to be analysable. Policies are versioned, reviewed and unit-tested. For speed, run the engine as a sidecar or library next to the service and cache decisions briefly.

5 ReBAC and Zanzibar

Sharing products (Google Drive, GitHub) need **relationship-based access control**. Google's Zanzibar paper (USENIX ATC 2019) stores relationship tuples `object#relation@subject` and answers checks by walking the graph. It reports trillions of tuples, millions of checks per second and p95 latency under 10 ms. Consistency tokens ("zookies") make sure a check never uses data older than a recent permission change.

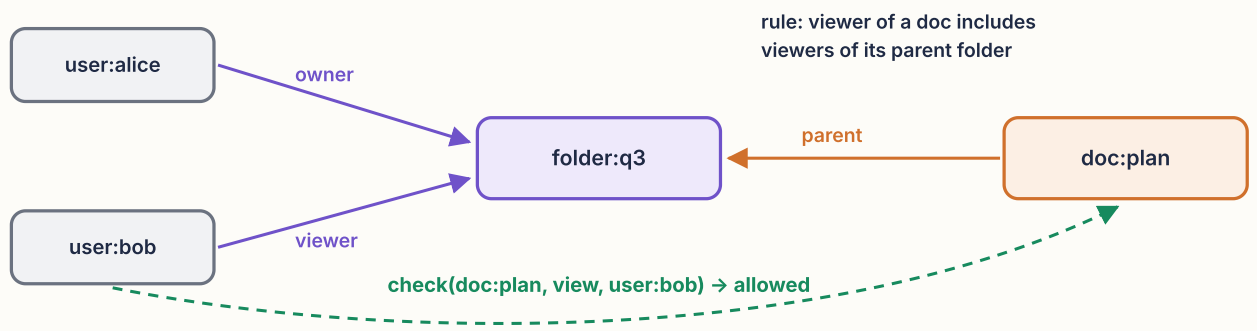


Figure 2. Bob was never granted the document; he can view it because he can view its parent folder.

	RBAC	ABAC	ReBAC
Access from	roles	attribute rules	relationship graph
Best for	admin panels, internal tools	context rules: region, time, device	sharing, folders, orgs, teams
Weakness	role explosion	hard to read and audit	needs a tuple store kept in sync
Tools	IAM, K8s RBAC	OPA, Cedar, XACML	SpiceDB, OpenFGA, Zanzibar

WORKED EXAMPLE: DOCUMENT SHARING WITH TUPLES

`folder:q3#owner@user:alice` · `folder:q3#viewer@user:bob` · `doc:plan#parent@folder:q3`. The schema says a doc's viewers include its parent's viewers. Bob opens the new doc: the check walks doc → parent folder → Bob is a viewer → **allowed**. Nobody wrote a role or rule for that document, and removing Bob's folder tuple revokes everything inside at once. SpiceDB and OpenFGA implement this model.

COMMON MISTAKES

- One role per customer or project until nobody knows what a role means.
- Checking only the role ("is editor") and not the resource ("editor of this project").
- Authorization logic copied into every service, slowly diverging.
- Untested policies: one bad rule silently allows or denies everything.

6 Key terms

RBAC: permissions granted through roles

ReBAC: access derived from relationships in a graph

PDP / PEP: policy decision point / enforcement point

ABAC: rules over subject, resource, action and environment attributes

Role explosion: too many narrow roles to manage

Relationship tuple: object#relation@subject, e.g. doc:plan#viewer@user:bob

7 Interview questions

Q1. When does RBAC break down? BASIC

When access depends on the specific resource, a relationship or context (time, region, device); teams create ever more specific roles until nobody understands them: role explosion.

Q2. What attributes does an ABAC rule use? BASIC

Subject (department, clearance), resource (owner, classification), action, and environment (time, IP, device posture).

Q3. Design permissions for a Google-Drive-like product. SCENARIO

ReBAC: store tuples for ownership, sharing, group membership and folder parents; checks walk the graph, so sharing a folder shares its contents. Use SpiceDB or OpenFGA rather than building it.

Q4. Ten services each implement their own permission checks. What do you change? SCENARIO

Move policies into a policy engine (OPA or Cedar) as versioned, tested code; services call it locally (sidecar or library) and enforce its decision.

Q5. How does Google Zanzibar model permissions? TRICKY

As relationship tuples (object#relation@subject) between users, groups and objects; a check follows the graph, with consistent, fast checks at huge scale.

Q6. What is the “new enemy” problem in Zanzibar? TRICKY

A check evaluated on stale data could let a just-removed user see content added after the removal. Zanzibar's zookies make checks at least as fresh as the content's version.

Next: SD-M-007 · Service-to-Service Auth backendroadmap.com